

Algebra and Normalisation

Ohad Kammar

slides



course
homepage



Zulip channel
#Algebra and normalisation



Scottish Programming Languages and Verification

Summer School (SPLV)

3–7 August 2026

University of Glasgow



THE UNIVERSITY OF EDINBURGH

informatics **ifcs**

Laboratory for Foundations
of Computer Science



BayesCentre

Funded by:



THE ROYAL
SOCIETY

Normalisation in PL and Logic: what?

Goal

Identify equivalent syntactic representations

Example: equivalent programs

$x := 1;$
 $x := 2$

vs

$x := 2$

vs

if $x > 0$
 then $x := 2$
 else $x := 2$

Example: algebraic expressions

$$5 + x + (0 + (-6)) \quad \text{vs} \quad -1 + x$$

Example: λ -terms (not today)

$$(\lambda x : \mathbb{Z}. x + 1)(y - 3) \quad \text{vs} \quad -2 + y$$

Normalisation: why?

Proofs

Deal with fewer cases

Robust systems: type-checkers, compilers, and partial evaluators

Behave in the same way and compositionally for equivalent programs

Search and synthesis

Enumerate candidates over smaller search-space

Blind Rewriting (BRew)

Temptation:

- ▶ Represent abstract syntax tree (AST)
- ▶ Apply all transformations
- ▶ Everywhere
- ▶ Blindly

BRew Caveats

- ▶ **Wasteful** in time & space (NB: **E-graphs**)
- ▶ **Chaotic** and **non-robust**: type errors after tiny tweaks to source
- ▶ **Syntax-sensitive** or **limited** (e.g.: $x + y$ vs $y + x$)

Disclaimer!

Don't confuse **BRew** with **strategic** & **term rewriting**:

- ▶ Deep maths & time-tested properties:
confluence, **strong normalisation**, **Knuth-Bendix completion**, ...
- ▶ Deep connections to foundations of **logic**, **mathematics**, & **computation**
(e.g.: λ -calculus)

This course

Goal

relate **normalisation** and **modern algebra**

This course

Goal

relate **normalisation** and **modern algebra**

By the end of the course, you will be able to:

- ▶ Identify **free algebras** and **free extensions** with **normalisation** tasks.
- ▶ Phrase and prove **representation theorems**.
- ▶ Design **data-structures** for normalisation based on representation theorems.

This course comprises:

- ▶ 3 lecture hours.
- ▶ 3–6 independent self-study hours.
- ▶ 40 independent exercises hours.

So the majority of the learning will happen **after** SPLV.



course
URL



Zulip
URL

Goal

relate **normalisation** and **modern algebra**



course
URL



Zulip
URL

- ▶ Part 1: universal algebra
 - ▶ Motivation
 - ▶ Universal algebra basics: signatures & algebras
 - ▶ Modern algebra: homomorphisms & representation theorems
 - ▶ Expression normalisation: free models & equational logic
 - ▶ Modern partial evaluation: free extensions
 - ▶ Categorical algebra basics
- ▶ Part 2: second-order multi-sorted algebra
- ▶ ~~Part 3: second-order normalisation~~

Universal algebra basics

template:	$(X_s)_{s \in \text{Sort}}$ family of carrier sets	&	$(f_i : X_{s_1} \times \dots \times X_{s_n} \rightarrow X_r)_{i \in I}$ family of operations
instances:	$\mathbb{Z} := \{\dots, -2, -1, 0, 1, 2, \dots\}$ (Sort = {★}) integers	&	$(+) : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ addition $0 : \mathbb{1} := \{\star\} \rightarrow \mathbb{Z}$ zero $(\cdot) : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ multiplication $1 : \mathbb{1} \rightarrow \mathbb{Z}$ one

Universal algebra basics

template:	$(X_s)_{s \in \text{Sort}}$ family of carrier sets	$\& \left(f_i : X_{s_1} \times \dots \times X_{s_n} \rightarrow X_r \right)_{i \in I}$ family of operations	
instances:	$\mathbb{Z}$ $(\text{Sort} = \{\star\})$ integers	$\& \quad (+) : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ $0 : \mathbb{1} := \{\star\} \rightarrow \mathbb{Z}$	addition zero
$(\text{Sort} = \mathbb{N} \times \mathbb{N})$	$(\mathbb{M}_{m \times n})_{m,n \in \mathbb{N}}$ real matrices	$\& \quad (\cdot) : \mathbb{M}_{m \times k} \times \mathbb{M}_{k \times n} \rightarrow \mathbb{M}_{m \times n}$ $\mathbf{I}_n : \mathbb{1} \rightarrow \mathbb{M}_{n \times n}$	multiplication unit matrix

Universal algebra basics

template:	$(X_s)_{s \in \text{Sort}}$ family of carrier sets	$\& \left(f_i : X_{s_1} \times \dots \times X_{s_n} \rightarrow X_r \right)_{i \in I}$ family of operations	
instances:	$\mathbb{Z}$	$\& (+) : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$	addition
(Sort = {★})	integers	$0 : \mathbb{1} := \{\star\} \rightarrow \mathbb{Z}$	zero
(Sort = $\mathbb{N} \times \mathbb{N}$)	$(M_{m \times n})_{m,n \in \mathbb{N}}$ real matrices	$\& (\cdot) : M_{m \times k} \times M_{k \times n} \rightarrow M_{m \times n}$	multiplication
		$I_n : \mathbb{1} \rightarrow M_{n \times n}$	unit matrix
	$\left(\Lambda_A := \left(\begin{array}{c} \text{closed} \\ \lambda\text{-terms} \\ \text{of type } A \end{array} \right) / \equiv_{\beta\eta} \right)_{A \in \text{SimpleType}}$ combinatory logic	$\& (@) : \Lambda_{A \rightarrow B} \times \Lambda_A \rightarrow \Lambda_B$	application
		$I := (\lambda x : A. x) : \mathbb{1} \rightarrow \Lambda_{A \rightarrow A}$	combinators
		$K := (\lambda x : A. \lambda y : B. x) : \mathbb{1} \rightarrow \Lambda_{A \rightarrow B \rightarrow A}$	
		$S := \left(\lambda x : A \rightarrow B \rightarrow C. \lambda y : A \rightarrow B. \lambda z : A. \right. \\ \left. x z (y z) \right) : \mathbb{1} \rightarrow \Lambda_{(\dots)}$	
⋮			

Universal algebra basics

Universal algebra

Study **common properties** of all such structures

(i.e.: universally applicable)

We'll do it in two steps:

- ▶ Generic syntax
- ▶ Generic interpreters

Running examples

Lecture focus on $\mathbb{Z}$ and $\mathbb{M}$

Exercises explore other examples similarly

Formal syntax

Finitary algebraic signature $S = (\text{Sort}_S, \text{Op}_S, \text{Arity}_S)$ (NB: indexed containers [Altenkirch et al. 14])

- ▶ **sort** set: $\text{Sort}_S \ni s$, written as decorated stars, e.g.: $\star_s$
- ▶ **operator** set: $\text{Op}_S \ni f$
- ▶ **arity** assignment $\text{Arity}_S : \text{Op}_S \rightarrow (\text{List } \text{Sort}_S) \times \text{Sort}_S$

Instead of $\text{Arity}_S f = ([s_1, \dots, s_n], r)$ write $(f : \star_{s_1} \times \dots \times \star_{s_n} \rightarrow \star_r) \in S$

Example: magma

- ▶ $\text{Sort}_{\text{Magma}} := \{_ \}$ (i.e.: underline)
- ▶ $\text{Op}_{\text{Magma}} := \{(*), 1\}$
- ▶ $\text{Arity}_{\text{Magma}}(*) := ([_, _], _)$
so $((*) : \star \times \star \rightarrow \star) \in \text{Magma}$
- ▶ $\text{Arity } 1 = ([], _)$ so $1 : \star$

Succinctly: $\text{Magma} = \{(*): \star \times \star \rightarrow \star, 1 : \star\}$

Example: reflexive quivers over set X

- ▶ $\text{Sort}_{X\text{-Quiver}} := \{\star_{m \times k} \mid m, k \in X\}$
- ▶ $X\text{-Quiver} :=$
 $\{(\cdot_{m,k,n}) : \star_{m \times k} \times \star_{k \times n} \rightarrow \star_{m \times n} \mid m, k, n \in X\}$
 $\cup \{I_n : \star_{n \times n} \mid n \in X\}$

Interpretation

Algebra $\mathbf{A} = \left((A_s)_{s \in \text{Sort}}, (f_A)_{f \in \text{Op}} \right)$ for signature $\mathbf{S}$

- ▶ Sort-indexed family of **carrier** sets: A_s for every $s \in \text{Sort}$.
- ▶ Op-indexed family of **operations** between the corresponding carriers:
 $f_A : A_{s_1} \times \dots \times A_{s_n} \rightarrow A_r$ for every operator $(f : \star_{s_1} \times \dots \times \star_{s_n} \rightarrow \star_r) \in \mathbf{S}$

Example: magmas

algebras $\mathbf{A}$ for the signature

Magma =

$\{ (* : \underline{\star} \times \underline{\star} \rightarrow \underline{\star}), 1 : \underline{\star} \}$

amount to:

- ▶ carrier set $\underline{A}$
- ▶ operation $(*_A) : \underline{A}^2 \rightarrow \underline{A}$
- ▶ constant $1_A \in \underline{A}$

Example: quivers

sets of matrices as algebra $\mathbb{M}$ for

N-Quiver :=

$\{ (\cdot_{m,k,n}) : \star_{m \times k} \times \star_{k \times n} \rightarrow \star_{m \times n} \mid m, k, n \in \mathbb{N} \} \cup \{ \mathbf{I}_n \mid n \in \mathbb{N} \}$

- ▶ carriers: $\mathbb{M}_{m \times n}$ (m rows, n columns)
- ▶ multiplication: $(a_{i,j})_{i,j} \cdot (b_{j,\ell})_{j,\ell} := \sum_j a_{i,j} \cdot b_{j,\ell}$
- ▶ identity matrix: $\mathbf{I}_n := \begin{pmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \dots & 0 & 1 \end{pmatrix}$

Algebraic expressions

Algebras give **meta-level** expressions:

Example: magma $\underline{A} = (\underline{A}, (\cdot) : \underline{A}^2 \rightarrow \underline{A}, 0)$

$$(x \cdot y) \cdot z \quad x \cdot (y \cdot z) \quad y \cdot (1 \cdot x) \quad (x, y, z \in \underline{A})$$

Example: $\mathbb{N}$ -Quiver of matrices $\mathbb{M}$

$$x \cdot (y \cdot z) \quad x \cdot (y \cdot z) \quad x \cdot (\mathbf{I}_k \cdot y) \quad (x \in \mathbb{M}_{n \times k}, y \in \mathbb{M}_{k \times \ell}, z \in \mathbb{M}_{\ell \times p})$$

Summary

We formulated the template and its instances:

- ▶ **Signatures** specify the operators and their argument and return **sorts**
- ▶ **Algebras** interpret these operators as operations of appropriate type

Both let us talk about our examples, and many other examples, generically.

Typically, consider sub-class of algebras **validating** axioms (e.g., **associativity**).
We cover these **varieties** later.

Goal

relate **normalisation** and **modern algebra**



course
URL



Zulip
URL

- ▶ Part 1: universal algebra
 - ▶ Motivation
 - ▶ Universal algebra basics: signatures & algebras
 - ▶ Modern algebra: homomorphisms & representation theorems
 - ▶ Expression normalisation: free models & equational logic
 - ▶ Modern partial evaluation: free extensions
 - ▶ Categorical algebra basics
- ▶ Part 2: second-order multi-sorted algebra
- ▶ ~~Part 3: second-order normalisation~~

The hallmarks of modern algebra are:

- ▶ studying **varieties** of algebras defined through **equational axioms**; and
- ▶ establishing **representation theorems** relating structures.

Both aspects pertinent for normalisation:

- ▶ The same **signature** can express many varieties of algebras.
It is the variety that determines **normalisation**.
- ▶ The same **family** may carry multiple algebraic structures.
The variety determines normalisation of **residual programs**.
- ▶ Representation theorems expose structural **properties**.

Structure-preserving-maps—**homomorphism**—are core to these relationship.

Structure-preserving maps

Homomorphism $h : A \rightarrow B$

between S -algebras A, B :

- ▶ indexed family of functions $h_s : A_s \rightarrow B_s$, for all $s \in \text{Sort}$;
- ▶ such that:

$$h_r(f_A(a_1, \dots, a_n)) = f_B(h_{s_1}a_1, \dots, h_{s_n}a_n)$$

for all $(f : \star_{s_1} \times \dots \times \star_{s_n} \rightarrow \star_r) \in S$ and $a_i \in A_{s_i}$, for $i = 1, \dots, n$

Examples

- ▶ Exponentiation is a magma homomorphism: $(\lambda x. 2^x) : (\mathbb{Z}, (+), 0) \rightarrow (\mathbb{Q}, (\cdot), 1)$
- ▶ Uniform base-change is a quiver homomorphism:

$$(\lambda T : \mathbb{M}_{m \times n} \cdot J_m \cdot T \cdot J_n^{-1})_{m,n} : \mathbb{M} \rightarrow \mathbb{M}$$

for family of invertible matrices $(J_n)_{n \in \mathbb{N}}$. (exercise: generalise)

Abstract syntax

Abstract syntax provides **object-level** terms, e.g., for first-class axioms
It'll be our first representation theorem:

$\text{Term}_{\mathbf{S}}X$

over signature $\mathbf{S}$ and **Sort**-indexed family variables X — **Sort**-indexed inductive family:

$$\frac{}{x \in (\text{Term}_{\mathbf{S}}X)_s} (x \in X_s) \quad \frac{t_i \in (\text{Term}_{\mathbf{S}}X)_{s_i} \text{ for all } i = 1, \dots, n}{f(t_1, \dots, t_n) \in (\text{Term}_{\mathbf{S}}X)_r} ((f : \star_{s_1} \times \dots \times \star_{s_n} \rightarrow \star_r) \in \mathbf{S})$$

Write $X \vdash_{\mathbf{S}} t : \star_s$ when $t \in (\text{Term}_{\mathbf{S}}X)_s$, and $(x : s) \in X$ when $x \in X_s$.

Data structure representing **abstract syntax trees** with $\mathbf{S}$ -operators and X -variables.

Abstract syntax as an abstract data type

The operations available on the abstract syntax are:

- ▶ form a **variable**;
- ▶ form a composite **term** using an operator and appropriately sorted sub-terms;
- ▶ eliminate a term by **traversal**, given:

- ▶ Motive $\underline{A}_s$ for every sort;
- ▶ Variable case: $e : X \rightarrow \underline{A}$
- ▶ Visitor for every AST node:
 $f_A : \underline{A}_{s_1} \times \dots \times \underline{A}_{s_n} \rightarrow \underline{A}_r$

$h := \text{fold}(\underline{A}, e, (f_A)_f) : \text{Term}_S X \rightarrow \underline{A}$

uniquely defined by:

$$h(\text{var } x) = e_s x \quad h(f(t_1, \dots, t_n)) = f_A(h t_1, \dots, h t_n)$$

Example

$S := \text{Magma}$ $\underline{X} := \{x, y, z\}$ $\underline{A} := (\mathbb{Z}, (+), 0)$

$e : X = \{x, y, z\} \rightarrow \mathbb{Z}$

$x \mapsto 10$ $y \mapsto 5$ $z \mapsto -6$ $5+10$ $0+(-6)$

$$\text{fold } \underline{A} \, e((y \cdot x) \cdot (1 \cdot z)) = (e y + e x) + (0 + e z) = 9$$

Abstract syntax as an abstract data type

The operations available on the abstract syntax are:

- ▶ form a **variable**;
i.e., a family of functions $\text{var} : X \rightarrow \text{Term}_S X$
- ▶ form a composite **term** using an operator and appropriately sorted sub-terms;
i.e., S -algebra structure $F_S X$:

$$(F_S X)_s := (\text{Term}_S X)_s \quad f_{F_S X} : (F_S X)_{s_1} \times \cdots \times (F_S X)_{s_n} \rightarrow (F_S X)_r$$

- ▶ eliminate a term by **traversal**, given:

- ▶ Motive $\underline{A}_s$ for every sort;

- ▶ Variable case: $e : X \rightarrow \underline{A}$

- ▶ Visitor for every AST node:

$$f_A : \underline{A}_{s_1} \times \cdots \times \underline{A}_{s_n} \rightarrow \underline{A}_r$$

$$h := \text{fold}(\underline{A}, e, (f_A)_f) : \text{Term}_S X \rightarrow \underline{A}$$

uniquely defined by:

$$h(\text{var } x) = e_s x \quad h(f_{F_S X}(t_1, \dots, t_n)) = f_A(h t_1, \dots, h t_n)$$

- ▶ i.e., each S -algebra $A = (\underline{A}, (f_A))$;

- ▶ and Sort -family map $e : X \rightarrow \underline{A}$

uniquely define:

- ▶ **homomorphism** $\text{fold } A e : F_S X \rightarrow A$

- ▶ s.t. $\text{fold } A$ **extends** $e : X \rightarrow \underline{A}$ along $\text{var} : X \rightarrow \underline{F_S X}$.

Abstract syntax as a mathematical structure

Focus on the underlying maths:



a family of functions $\text{var} : X \rightarrow \text{Term}_S X$



S -algebra structure $F_S X$:

$$(F_S X)_s := (\text{Term}_S X)_s \quad f_{F_S X} : (F_S X)_{s_1} \times \cdots \times (F_S X)_{s_n} \rightarrow (F_S X)_r$$



▶ i.e., each S -algebra $A = (\underline{A}, (f_A))$;

▶ and Sort -family map $e : X \rightarrow \underline{A}$

uniquely define:

▶ **homomorphism** $\text{fold } A \, e : F_S X \rightarrow A$

▶ s.t. $\text{fold } A \, \text{extends } e : X \rightarrow \underline{A}$ along $\text{var} : X \rightarrow \underline{F_S X}$.

Abstract syntax as a mathematical structure

Focus on the maths, rearrange into the situation:

- ▶ $F_S X$ is a **S**-algebra:

$$(F_S X)_s := (\text{Term}_S X)_s \quad f_{F_S X} : (F_S X)_{s_1} \times \cdots \times (F_S X)_{s_n} \rightarrow (F_S X)_r$$

- ▶ together with a family of functions $\text{var} : X \rightarrow \text{Term}_S X$

- ▶ For every:

- ▶ **S**-algebra $A = (\underline{A}, (f_A))$

- ▶ **Sort**-family map $e : X \rightarrow \underline{A}$

uniquely defines:

- ▶ a **homomorphism** $\text{fold } A \, e : F_S X \rightarrow A$

- ▶ s.t. **fold** A **extends** $e : X \rightarrow \underline{A}$ along $\text{var} : X \rightarrow \underline{F_S X}$.

Underlying abstraction:

algebra over X :

- ▶ **S**-algebra $B = (\underline{B}, (f_B))$

- ▶ **Sort**-family map $d : X \rightarrow \underline{B}$, the **environment/valuation**

and their **maps** $h : (B, d) \rightarrow (A, e)$:

- ▶ homomorphism $h : B \rightarrow A$

- ▶ preserving the environments:
 h extends e along d : $e = h \circ d$

Abstract syntax as a mathematical structure

Focus on the maths, use underlying abstraction:

- ▶ $F_S X$ is a **S**-algebra:

$$(F_S X)_s := (\text{Term}_S X)_s \quad f_{F_S X} : (F_S X)_{s_1} \times \cdots \times (F_S X)_{s_n} \rightarrow (F_S X)_r \\ \text{var} : X \rightarrow \text{Term}_S X$$

- ▶ For every (A, e) **S**-algebra over X , we have a map:

$$\text{fold } A e : (F_S X, \text{var}) \rightarrow (A, e)$$

Underlying abstraction:

algebra over X :

- ▶ **S**-algebra $B = (\underline{B}, (f_B))$
- ▶ **Sort**-family map $d : X \rightarrow \underline{B}$, the **environment/valuation**

and their **maps** $h : (B, d) \rightarrow (A, e)$:

- ▶ homomorphism $h : B \rightarrow A$
- ▶ preserving the environments:
 h extends e along d : $e = h \circ d$

Representation theorem for abstract syntax

Our first **representation theorem**:

Theorem (abstract syntax initiality)

Let $\mathbf{S}$ be a signature, and X a $\text{Sort}_{\mathbf{S}}$ -family. Then:

- ▶ abstract syntax trees for a signature $\mathbf{S}$ over a $\text{Sort}_{\mathbf{S}}$ -family of variables X ;
- ▶ its $\mathbf{S}$ -algebra structure $\mathbf{F}_{\mathbf{S}}X$:

$$(\mathbf{F}_{\mathbf{S}}X)_s := (\text{Term}_{\mathbf{S}}X)_s \quad f_{\mathbf{F}_{\mathbf{S}}X} : (\mathbf{F}_{\mathbf{S}}X)_{s_1} \times \cdots \times (\mathbf{F}_{\mathbf{S}}X)_{s_n} \rightarrow (\mathbf{F}_{\mathbf{S}}X)_r$$

- ▶ the variable map $\text{var} : X \rightarrow \mathbf{F}_{\mathbf{S}}X$

form a **free $\mathbf{S}$ -algebra over X** , i.e. an initial $\mathbf{S}$ -algebra over X :

for every $\mathbf{S}$ -algebra $(\mathbf{A}, e)$ over X there is a unique map $\text{fold } \mathbf{A} \, e : (\mathbf{F}_{\mathbf{S}}X, \text{var}) \rightarrow (\mathbf{A}, e)$.

Succinctly— $\mathbf{A}$ -natural bijection:

$$\frac{e : X \longrightarrow \mathbf{A}}{\text{fold } \mathbf{A} \, e : \mathbf{F}_{\mathbf{S}}X \longrightarrow \mathbf{A}} \quad \begin{array}{c} \text{recover universal arrow as} \\ \text{mate of identity:} \end{array} \quad \frac{\text{var} : X \longrightarrow \mathbf{F}_{\mathbf{S}}X = \text{Term}_{\mathbf{S}}X}{\text{id} : \mathbf{F}_{\mathbf{S}}X \longrightarrow \mathbf{F}_{\mathbf{S}}X}$$

Algebra and Normalisation: Lecture 2

Ohad Kammar

slides



course
homepage



Zulip channel
#Algebra and normalisation



Scottish Programming Languages and Verification

Summer School (SPLV)

3–7 August 2026

University of Glasgow



THE UNIVERSITY OF EDINBURGH

informatics **ifcs**

Laboratory for Foundations
of Computer Science



BayesCentre

Funded by:



THE ROYAL
SOCIETY

Recap: universal algebra basics

Universal algebra

We formulated the template and its instances:

- **Signatures** specify the operators and their argument and return **sorts**

$$\text{E.g.: Magma} := \{(\cdot) : \underline{\star} \times \underline{\star} \rightarrow \underline{\star}, 1 : \underline{\star}\}$$

$$\text{Generally: } S = (\text{Sort} \in \text{Set}, \text{Op} \in \text{Set}, \text{Arity} : \text{Op} \rightarrow \text{List Sort} \times \text{Sort})$$

- **Algebras** interpret these operators as operations of appropriate type:

$$\text{E.g., Magma-algebra: } A = (\underline{A}, (\cdot) : \underline{A}^2 \rightarrow \underline{A}, 1 : \underline{A})$$

$$\text{Generally: } A = \left((\underline{A}_s)_{s \in \text{Sort}} \in \mathbf{Fam} \text{ Sort}, \left(f_A : \underline{A}_{s_1} \times \cdots \times \underline{A}_{s_n} \rightarrow \underline{A}_r \right)_{(f : \underline{\star}_{s_1} \times \cdots \times \underline{\star}_{s_n} \rightarrow \underline{\star}_r) \in S} \right)$$

Both let us talk about our examples, and many other examples, generically.

Abstract syntax

Abstract syntax provides **object-level** terms, e.g., for first-class axioms
It'll be our first representation theorem:

$\text{Term}_{\mathbf{S}}X$

over signature $\mathbf{S}$ and **Sort**-indexed family variables X — **Sort**-indexed inductive family:

$$\frac{}{x \in (\text{Term}_{\mathbf{S}}X)_s} (x \in X_s) \quad \frac{t_i \in (\text{Term}_{\mathbf{S}}X)_{s_i} \text{ for all } i = 1, \dots, n}{f(t_1, \dots, t_n) \in (\text{Term}_{\mathbf{S}}X)_r} ((f : \star_{s_1} \times \dots \times \star_{s_n} \rightarrow \star_r) \in \mathbf{S})$$

Write $X \vdash_{\mathbf{S}} t : \star_s$ when $t \in (\text{Term}_{\mathbf{S}}X)_s$, and $(x : s) \in X$ when $x \in X_s$.

Data structure representing **abstract syntax trees** with $\mathbf{S}$ -operators and X -variables.

Representation theorem for abstract syntax

Our first **representation theorem**:

Theorem (abstract syntax initiality)

Let $\mathbf{S}$ be a signature, and X a $\text{Sort}_{\mathbf{S}}$ -family. Then:

- ▶ abstract syntax trees for a signature $\mathbf{S}$ over a $\text{Sort}_{\mathbf{S}}$ -family of variables X ;
- ▶ its $\mathbf{S}$ -algebra structure $\mathbf{F}_{\mathbf{S}}X$:

$$(\mathbf{F}_{\mathbf{S}}X)_s := (\text{Term}_{\mathbf{S}}X)_s \quad f_{\mathbf{F}_{\mathbf{S}}X} : (\mathbf{F}_{\mathbf{S}}X)_{s_1} \times \cdots \times (\mathbf{F}_{\mathbf{S}}X)_{s_n} \rightarrow (\mathbf{F}_{\mathbf{S}}X)_r$$

- ▶ the variable map $\text{var} : X \rightarrow \mathbf{F}_{\mathbf{S}}X$

form a **free $\mathbf{S}$ -algebra over X** , i.e. an initial $\mathbf{S}$ -algebra over X :

for every $\mathbf{S}$ -algebra $(\mathbf{A}, e)$ over X there is a unique map $\text{fold } \mathbf{A} \, e : (\mathbf{F}_{\mathbf{S}}X, \text{var}) \rightarrow (\mathbf{A}, e)$.

Succinctly— $\mathbf{A}$ -natural bijection:

$$\frac{e : X \longrightarrow \mathbf{A}}{\text{fold } \mathbf{A} \, e : \mathbf{F}_{\mathbf{S}}X \longrightarrow \mathbf{A}} \quad \begin{array}{c} \text{recover universal arrow as} \\ \text{mate of identity:} \end{array} \quad \frac{\text{var} : X \longrightarrow \mathbf{F}_{\mathbf{S}}X = \text{Term}_{\mathbf{S}}X}{\text{id} : \mathbf{F}_{\mathbf{S}}X \longrightarrow \mathbf{F}_{\mathbf{S}}X}$$

Representation theorems for abstract syntax

- ▶ The representation theorem isolates the **interface** to abstract syntax
- ▶ $(F_S X)_s := (Term_S X)_s$ is one implementation

Another implementation: “concrete” syntax

Given coproduct diagram:

$$Op_S \xrightarrow{Op} Symbol \xleftarrow{Var} X$$

Define $(C_S X)_s \subseteq List\ Symbol$:

$$\frac{}{[Var x] \in (C_S X)_s} (x \in X_s) \quad \frac{a_i \in (C_S X)_{s_i} \text{ for all } i = 1, \dots, n}{(Op f :: a_1 \# \dots \# a_n) \in (C_S X)_r} ((f : \star_{s_1} \times \dots \times \star_{s_n} \rightarrow \star_r) \in S)$$

equip it with S -algebra structure over X :

$$(G_S X)_s := (C_S X)_s \quad f_{G_S X}(a_1, \dots, a_n) := (Op f :: a_1 \# \dots \# a_n) \quad var_s x := [Var x]$$

Representation theorems for abstract syntax

Theorem (“concrete” syntax initiality)

Let $\mathbf{S}$ be a signature, and X a $\text{Sort}_{\mathbf{S}}$ -family. Take Symbol , $\mathbf{C}_{\mathbf{S}}X$ from previous slide. The following $(\mathbf{G}_{\mathbf{S}}X, \text{var})$:

$$(\mathbf{G}_{\mathbf{S}}X)_s := (\mathbf{C}_{\mathbf{S}}X)_s \quad f_{\mathbf{G}_{\mathbf{S}}X}(a_1, \dots, a_n) := (\text{Op}f :: a_1 \# \dots \# a_n) \quad \text{var}_s x := [\text{Var}x]$$

is a free $\mathbf{S}$ -algebra over X .

(Exercise sheet explores this implementation and others in more detail.)

Goal

relate **normalisation** and **modern algebra**



course
URL



Zulip
URL

- ▶ Part 1: universal algebra
 - ▶ Motivation
 - ▶ Universal algebra basics: signatures & algebras
 - ▶ Modern algebra: homomorphisms & representation theorems
 - ▶ Expression normalisation: free models & equational logic
 - ▶ Modern partial evaluation: free extensions
 - ▶ Categorical algebra basics
- ▶ Part 2: second-order multi-sorted algebra
- ▶ ~~Part 3: second-order normalisation~~

Main takeaway

Core idea: algebra and normalisation

representation theorem provides **normalisation** procedure

Already appears in **abstract syntax**—different concrete representations

$"(x_ + _y)"$ vs. $"x + y"$
6 characters 3 characters

reflected by same abstract representation:

$x + y$ i.e.: $\begin{cases} \text{tree representation: } (+)(x, y) \\ \text{"concrete" representation: } [\text{Op } (+), \text{Var } x, \text{Var } y] \end{cases}$

reifying the representation as a string gives the **same** concrete syntax, say:

$"x + y"$

We'll unpack this for **monoids**.

Monoid representation

Monoid $\underline{A} = (\underline{A}, (\cdot) : \underline{A}^2 \rightarrow \underline{A}, 1 : \underline{A}^2)$

associative unital magma:

► **associative:**

$$(x \cdot y) \cdot z = x \cdot (y \cdot z) \quad (x, y, z \in \underline{A})$$

► **unital:**

$$x \cdot 1 = x = 1 \cdot x \quad (x \in \underline{A})$$

monoid homomorphism $h : \underline{A} \rightarrow \underline{B}$ is a magma homomorphism

Monoid $\underline{A}$ over set X

► function $d : X \rightarrow \underline{A}$, the **environment**

and thier maps $h : (\underline{A}, d) \rightarrow (\underline{B}, e)$:

► homomorphism $h : \underline{A} \rightarrow \underline{B}$

► preserving the environment: h extends e along d .

Monoid representation

Theorem

Let X be a set. The following data defines a free monoid over X :

$$\underline{F_{\text{Monoid}} X} := \text{List } X \quad u \cdot v := u \text{ ++ } v \quad 1 := [] \quad \eta^{\text{Monoid}} x := [x]$$

i.e.:

- ▶ $F_{\text{Monoid}} X := (\text{List } X, (+), [])$ is a monoid, and
- ▶ for every $e : X \rightarrow \underline{A}$ monoid A over X there is a unique monoid homomorphism:

$$\text{eval } A e : F_{\text{Monoid}} X \rightarrow A$$

extending e along the unit η^{Monoid} . It is given by:

$$\text{eval } A e [x_1, \dots, x_n] := 1 \cdot (e x_1) \cdot \dots \cdot (e x_n) \quad \frac{e : X \longrightarrow \underline{A} \text{ in Set}}{\text{eval } A e : F_{\text{Monoid}} X \longrightarrow A \text{ in Monoid}}$$

Core idea: algebra and normalisation

representation theorem provides **normalisation** procedure

We have a representation theorem, so we deserve a normalisation procedure:

Theorem (extensional normalisation, informally)

Let $(F_{\text{Monoid}} X, \eta^{\text{Monoid}})$ be a free monoid over the set X , and t, s be monoid expressions with variables from X .

The equation $t = u$ is valid in all monoids iff it is valid in $F_{\text{Monoid}} X$.

To phrase and prove this theorem precisely, we need to define **validity** of equations.

Equation $X \vdash_S t = u : \star_s$ over signature S

- ▶ set X
- ▶ terms $t, s \in (\text{Term}_S X)_s$

Satisfaction $(A, e) \models (X \vdash t = u : \star_s)$

by S -algebra $e : X \rightarrow \underline{A}$ over X :

$$\text{fold } A \, e \, t = \text{fold } A \, e \, u$$

Algebra A **validates** equation $A \models (X \vdash t = u : \star_s)$ when every environment satisfies it:

$$\forall e : X \rightarrow \underline{A}. \quad (A, e) \models (X \vdash t = u : \star_s)$$

Satisfaction $(\mathbf{A}, e) \models (X \vdash t = u : \star_s)$

by $\mathbf{S}$ -algebra $e : X \rightarrow \underline{\mathbf{A}}$ over X :

$$\text{fold } \mathbf{A} \, e \, t = \text{fold } \mathbf{A} \, e \, u$$

Algebra $\mathbf{A}$ **validates** equation $\mathbf{A} \models (X \vdash t = u : \star_s)$ when every environment satisfies it:

$$\forall e : X \rightarrow \underline{\mathbf{A}}. \quad (\mathbf{A}, e) \models (X \vdash t = u : \star_s)$$

Examples

- The unit of the list monoid satisfies the equation:

$$(\mathbf{F}_{\text{Monoid}} \{x, y\}, \eta^{\text{Monoid}}) \models (\{x, y\} \vdash (x \cdot 1) \cdot y = 1 \cdot (x \cdot y) : \underline{\star})$$

since:

$$([x] \# []) \# [y] = [x, y] = [] \# ([x] \# [y])$$

Validity

Satisfaction $(\mathbf{A}, e) \models (X \vdash t = u : \star_s)$

by $\mathbf{S}$ -algebra $e : X \rightarrow \underline{\mathbf{A}}$ over X :

$$\text{fold } \mathbf{A} \, e \, t = \text{fold } \mathbf{A} \, e \, u$$

Algebra $\mathbf{A}$ **validates** equation $\mathbf{A} \models (X \vdash t = u : \star_s)$ when every environment satisfies it:

$$\forall e : X \rightarrow \underline{\mathbf{A}}. \quad (\mathbf{A}, e) \models (X \vdash t = u : \star_s)$$

Examples

- In fact, every monoid validates the equation:

$$\mathbf{A} \models (\{x, y\} \vdash (x \cdot \mathbf{1}) \cdot y = \mathbf{1} \cdot (x \cdot y) : \underline{\star})$$

since:

$$\begin{array}{ccc} & \text{unitality} & \text{unitality} \\ & \downarrow & \downarrow \\ ((e \, x) \cdot \mathbf{1}) \cdot (e \, y) & = & (e \, x) \cdot (e \, y) = \mathbf{1} \cdot ((e \, x) \cdot (e \, y)) \end{array}$$

for every environment $e : \{x, y\} \rightarrow \underline{\mathbf{A}}$

Satisfaction $(\mathbf{A}, e) \models (X \vdash t = u : \star_s)$

by $\mathbf{S}$ -algebra $e : X \rightarrow \underline{\mathbf{A}}$ over X :

$$\text{fold } \mathbf{A} \, e \, t = \text{fold } \mathbf{A} \, e \, u$$

Algebra $\mathbf{A}$ **validates** equation $\mathbf{A} \models (X \vdash t = u : \star_s)$ when every environment satisfies it:

$$\forall e : X \rightarrow \underline{\mathbf{A}}. \quad (\mathbf{A}, e) \models (X \vdash t = u : \star_s)$$

Examples

- ▶ A monoid is a **Magma**-algebra $\mathbf{A}$ validating the associativity and unitality:

$$\mathbf{A} \models (\{x, y, z\} \vdash (x \cdot y) \cdot z = x \cdot (y \cdot z) : \underline{\star}) \quad \mathbf{A} \models (\{x\} \vdash x \cdot \mathbf{1} = x = \mathbf{1} \cdot x : \underline{\star})$$

Validity

Satisfaction $(\mathbf{A}, e) \models (X \vdash t = u : \star_s)$

by $\mathbf{S}$ -algebra $e : X \rightarrow \underline{\mathbf{A}}$ over X :

$$\text{fold } \mathbf{A} \, e \, t = \text{fold } \mathbf{A} \, e \, u$$

Algebra $\mathbf{A}$ **validates** equation $\mathbf{A} \models (X \vdash t = u : \star_s)$ when every environment satisfies it:

$$\forall e : X \rightarrow \underline{\mathbf{A}}. \quad (\mathbf{A}, e) \models (X \vdash t = u : \star_s)$$

Lemma (homomorphic images preserve equations)

Let $h : \mathbf{A} \rightarrow \mathbf{B}$ be an $\mathbf{S}$ -homomorphism. Then:¹

$$(\mathbf{A}, e) \models (X \vdash t = u : \star_s) \implies (\mathbf{B}, h \circ e) \models (X \vdash t = u : \star_s)$$

So if $h : \underline{\mathbf{A}} \rightarrow \underline{\mathbf{B}}$ is **surjective**, then: $\mathbf{A} \models (X \vdash t = u : \star_s) \implies \mathbf{B} \models (X \vdash t = u : \star_s)$.

¹The slides given in the lecture had a mistake here, thanks Orpheas van Rooij for the correction.

Theorem (extensional normalisation)

Let $(F_{\text{Monoid}}X, \eta^{\text{Monoid}})$ be a free monoid over X , and $X \vdash t = u : \underline{\star}$ an equation. The following are equivalent (TFAE):

1. The unit satisfies the equation: $(F_{\text{Monoid}}X, \eta^{\text{Monoid}}) \models (X \vdash t = u : \underline{\star})$
2. The free monoid validates the equation: $F_{\text{Monoid}}X \models (X \vdash t = u : \underline{\star})$
3. Every monoid $A \in \mathbf{Monoid}$ validates the equation: $A \models (X \vdash t = u : \underline{\star})$

Proof idea, implemented over next few slides

By suitable instantiation: $(1) \iff (2) \iff (3)$. For $(3) \iff (1)$:

- ▶ Develop proof theory for universal algebra: **equational logic**
- ▶ By **soundness** of equational logic, a provable equation is valid in all monoids
- ▶ The **quotient** of the free magma by **provability** is a **free monoid**. ■

Proving this theorem for monoids is as easy as proving it for every variety of algebras.

Varieties of algebras

Presentation $\mathcal{P} = (\mathbf{S}, \mathbf{Ax})$

- ▶ signature $\mathbf{S}$
- ▶ set $\mathbf{Ax}$ of $\mathbf{S}$ -equations $(X \vdash t = u) \in \mathbf{Ax}$, the **axioms**

$\mathcal{P}$ -model: $\mathbf{S}$ -algebra $\mathbf{A}$ validating every axiom $\mathbf{A} \models (X \vdash t = u : \star_s) \in \mathbf{Ax}$

The class of models for some presentation is called a **variety** of algebras.

Example: monoids

Monoid = $(\mathbf{Magma}, \mathbf{Ax}_{\mathbf{Monoid}})$ presentation:

$$\{x, y, z\} \vdash (x \cdot y) \cdot z = x \cdot (y \cdot z) : \underline{\star} \quad \{x\} \vdash x \cdot 1 = x = 1 \cdot x : \underline{\star}$$

monoids are **Monoid**-models

Quotienting an algebra

Congruence ($\equiv$) over S-algebra $\mathbf{A}$

Sort-indexed family of equivalence relations ($\equiv_s$) over $\mathbf{A}_s$ that respects the operations:

$$\frac{a \in \mathbf{A}_s}{a \equiv_s a} (\text{refl})$$

$$\frac{a \equiv_s b}{b \equiv_s a} (\text{sym})$$

$$\frac{a \equiv_s b \quad b \equiv_s c}{a \equiv_s c} (\text{trans})$$

$$\frac{X \vdash e_1 \equiv e_2 \quad X \vdash t : \star_s}{\text{fold } \mathbf{A} \, e_1 \, t \equiv_s \text{fold } \mathbf{A} \, e_2 \, t} (\text{cong})$$

where $X \vdash e_1 \equiv e_2$ for environments $e_1, e_2 : X \rightarrow \underline{\mathbf{A}}$ when:

$$e_1 \, x \equiv_s e_2 \, x \text{ for every } (x : \star_s) \in X.$$

Example: length parity over $\mathbf{F}_{\text{Monoid}} X = (\text{List } X, (\oplus), [])$.

$u \equiv_{\star} v$ when u, v both odd-length or both even-length is a congruence

Quotienting an algebra

Quotient $\mathbf{S}$ -algebra $\mathbf{A}/(\equiv)$

(where $(\equiv)$ congruence over $\mathbf{A}$)

$$(\mathbf{A}/(\equiv))_s := \mathbf{A}_s/(\equiv_s) \quad f_{\mathbf{A}/(\equiv)}([a_1]_{s_1}, \dots, [a_n]_{s_n}) := [f_{\mathbf{A}}(a_1, \dots, a_n)]_r \\ ((f : \star_{s_1} \times \dots \times \star_{s_n} \rightarrow s_r) \in \mathbf{S})$$

Lemma

The quotient map is a $\mathbf{S}$ -homomorphism $[-]_- : \mathbf{A} \rightarrow \mathbf{A}/(\equiv)$, it is the universal $(\equiv)$ -respecting homomorphism out of $\mathbf{A}$:

- ▶ for all $a \equiv_s b$, we have $[a]_s = [b]_s$;
- ▶ for every $h : \mathbf{A} \rightarrow \mathbf{B}$ satisfying $a \equiv_s b \implies h_s a = h_s b$, there is a unique homomorphism $\therefore h : \mathbf{A}/(\equiv) \rightarrow \mathbf{B}$ extending h along $[-]$.

Equational logic: preliminaries

A substitution $X \vdash_S \theta : Y$

family map $\text{Term}_S X \xleftarrow{\theta} Y$, i.e.: $(X \vdash \theta x : \star_s) \leftrightarrow (x : \star_s) \in Y$

Substitution operation (given $X \vdash_S \theta : Y$)

family map $-[\theta] : \text{Term}_S X \xleftarrow{\text{fold}(\text{F}_S X) \theta} \text{Term}_S Y$, i.e.:

$$(X \vdash t[\theta] : \star_s) \leftrightarrow (Y \vdash t : \star_s)$$

Example

Substitution does what you think it should:

$$((x \cdot y) \cdot (x \cdot 1)) \overbrace{[x \mapsto z, y \mapsto 1 \cdot z]}^{\theta :=} = ((\theta x) \cdot (\theta y)) \cdot ((\theta x) \cdot 1) = (z \cdot (1 \cdot z)) \cdot (z \cdot 1)$$

Equational logic

Provability $X \vdash_{\mathcal{P}} t = u : \star_s$

S-congruence closure of the axiom:

$$\frac{}{X \vdash_{\mathcal{P}} t = t : \star_s} \text{(refl)} \qquad \frac{X \vdash_{\mathcal{P}} t = u : \star_s}{X \vdash_{\mathcal{P}} u = t : \star_s} \text{(sym)}$$

$$\frac{X \vdash_{\mathcal{P}} t = u : \star_s \quad X \vdash_{\mathcal{P}} u = v : \star_s}{X \vdash_{\mathcal{P}} t = v : \star_s} \text{(trans)}$$

$$\frac{X \vdash_{\mathcal{P}} \theta_1 = \theta_2 : Y \quad Y \vdash_{\mathcal{S}} t : \star_s}{X \vdash_{\mathcal{P}} t[\theta_1] = t[\theta_2] : \star_s} \text{(cong)} \qquad \frac{(Y \vdash t = u : \star_s) \in \mathbf{Ax}_{\mathcal{P}} \quad X \vdash_{\mathcal{S}} \theta : Y}{X \vdash_{\mathcal{P}} t[\theta] = u[\theta] : \star_s} \text{(axiom)}$$

where $X \vdash_{\mathcal{P}} \theta_1 = \theta_2 : Y$ when $X \vdash \theta_1 x = \theta_2 x : \star_r$ for all $(x : \star_r) \in Y$.

Example

We write equational proof referring to axioms only:

$$\{x, y\} \vdash_{\text{Monoid}} (x \cdot \mathbf{1}) \cdot y = x \cdot y = \mathbf{1} \cdot (x \cdot y)$$

$\uparrow \quad \quad \uparrow$
 unitality unitality

but mean derivations:

$$\begin{array}{c}
 \dfrac{(\{x\} \vdash x \cdot \mathbf{1} = x) \in \mathbf{Ax}}{\{x, y\} \vdash x \cdot \mathbf{1} = x} \text{(axiom)} \quad \dfrac{}{y = y} \text{(refl)} \\
 \hline
 \{x, y\} \vdash (z \mapsto x \cdot \mathbf{1}, y \mapsto y) = (z \mapsto x, y \mapsto y) : \{z, y\} \\
 \hline
 (x \cdot \mathbf{1}) \cdot y = x \cdot y \quad \dfrac{(z = \mathbf{1} \cdot z) \in \mathbf{Ax}}{x \cdot y = \mathbf{1} \cdot (x \cdot y)} \text{(axiom)} \\
 \hline
 \{x, y\} \vdash (x \cdot \mathbf{1}) \cdot y = \mathbf{1} \cdot (x \cdot y) : \star \quad \text{(trans)}
 \end{array}$$

Relating proofs and models

Theorem (soundness)

For every $\mathcal{P}$ -model $\mathbf{A}$:

$$X \vdash_{\mathcal{P}} t = u : \star_s \implies \mathbf{A} \models (X \vdash t = u : \star_s)$$

Proof

By induction on the derivation of the provability relation.

- **reflexivity, symmetry, transitivity** use corresponding property of equality, e.g.:

$$(\mathbf{A}, e) \models t = u \implies \text{fold } \mathbf{A} \ e \ t = \text{fold } \mathbf{A} \ e \ u \implies \text{fold } \mathbf{A} \ e \ u = \text{fold } \mathbf{A} \ e \ t \implies (\mathbf{A}, e) \models u = t$$

- **congruence** uses Leibniz's law, and basic properties of substitutions:

$$\begin{array}{ccccccc} \text{fold } \mathbf{A} \ e \ (t \ [\theta_1]) & = & \text{fold } \mathbf{A} \ (\text{fold } \mathbf{A} \ e \circ \theta_1) \ (t) & = & \text{fold } \mathbf{A} \ (\text{fold } \mathbf{A} \ e \circ \theta_2) \ (t) & = & \text{fold } \mathbf{A} \ e \ (t \ [\theta_2]) \\ \uparrow & & \uparrow & & & & \\ \text{substitution lemma} & & \text{IH+Leibniz's law} & & \implies & & (\mathbf{A}, e) \models t \ [\theta_1] = t \ [\theta_2] \end{array}$$

- **axiom** uses the $\mathcal{P}$ -model assumption. ■

Free model by quotienting

Theorem (quotiented algebra representation)

Let $\mathcal{P}$ be a presentation and X a family. We have a free $\mathcal{P}$ -model over X :

$$\mathbf{F}_{\mathcal{P}}X := \mathbf{F}_{\mathbf{S}}X / (X \vdash (=)) \quad \eta^{\mathcal{P}} : X \xrightarrow{\text{var}} \mathbf{F}_{\mathbf{S}}X \xrightarrow{[-]} (\mathbf{F}_{\mathbf{S}}X / (=)) =: \underline{\mathbf{F}_{\mathcal{P}}X}$$

I.e., for every $\mathcal{P}$ -model $\mathbf{A}$ and family map $e : X \rightarrow \underline{\mathbf{A}}$ there is a unique $\mathbf{S}$ -homomorphism extending e along $\eta^{\mathcal{P}}$ given by $\therefore \text{fold } \mathbf{A} \, e : \mathbf{F}_{\mathcal{P}}X \rightarrow \mathbf{A}$.

Proof assembles existing parts:

$$\begin{array}{c} e : X \longrightarrow \underline{\mathbf{A}} \text{ family map} \\ \hline \hline \text{fold } \mathbf{A} \, e : \mathbf{F}_{\mathbf{S}}X \longrightarrow \mathbf{A} \text{ } \mathbf{S}\text{-homomorphism, by freeness} \\ \hline \hline \therefore \text{fold } \mathbf{A} \, e : \mathbf{F}_{\mathcal{P}}X := \mathbf{F}_{\mathbf{S}}X / (X \vdash (=)) \longrightarrow \mathbf{A} \text{ } \mathbf{S}\text{-homomorphism, by lemma} \end{array}$$

Normalisation via representability

Theorem (extensional normalisation)

Let $(F_P X, \eta^P)$ be a free $\mathcal{P}$ -model over a family X , and $X \vdash t = u : \underline{\star}$ an equation. The following are equivalent (TFAE):

1. The unit satisfies the equation: $(F_P X, \eta^P) \models (X \vdash t = u : \underline{\star})$
2. The given free $\mathcal{P}$ -model validates the equation: $F_P X \models (X \vdash t = u : \underline{\star})$
3. Every $\mathcal{P}$ -model A validates the equation: $A \models (X \vdash t = u : \underline{\star})$
4. The equation is provable from $\mathcal{P}$: $X \vdash_P t = u : \underline{\star}$

Proof

Suitable instantiation/soundness: $(1) \iff (2) \iff (3) \iff (4)$. For $(4) \iff (1)$:

- ▶ The free $\mathcal{P}$ -models are canonically isomorphic $h : (F_P X, \eta^P) \xrightarrow{\cong} (F_S X / (=), [\text{var}])$.
- ▶ Homo. images preserve equations $\implies (F_S X / (=), [\text{var}]) \models t = u \implies X \vdash_P t = u$

as we wanted. ■

Summary

Core idea: algebra and normalisation

representation theorem
for an **effective** algebra provides extensional **normalisation** procedure, i.e.:
decide equality

Refinement: **effective** algebra and normalisation

effective
representation theorem provides intensional **normalisation** procedure, i.e.:
for an **effective** algebra decide a derivation of every equation

- ▶ Prove a constructive representation theorem for the free **setoid** $\mathcal{P}$ -model.
- ▶ Evaluate the equation in the effective free algebra, and transport it to the setoid-quotient free $\mathcal{P}$ -model.

See *Frex: dependently-typed algebraic simplification*. Guillaume Allais, Edwin Brady, Nathan Corbyn, Ohad Kammar, and Jeremy Yallop. 2025. DOI: 10.1145/3747506.

Algebra and Normalisation: Lecture 3

Ohad Kammar

slides



course
homepage



Zulip channel
#Algebra and normalisation



Scottish Programming Languages and Verification

Summer School (SPLV)

3–7 August 2026

University of Glasgow



THE UNIVERSITY OF EDINBURGH

informatics **ifcs**

Laboratory for Foundations
of Computer Science



BayesCentre

Funded by:



THE ROYAL
SOCIETY

Recap: universal algebra

Presentation $\mathcal{P} = (\mathbf{S}, \mathbf{Ax})$

- ▶ signature $\mathbf{S}$
- ▶ set $\mathbf{Ax}$ of $\mathbf{S}$ -equations $(X \vdash t = u) \in \mathbf{Ax}$, the **axioms**

$\mathcal{P}$ -model: $\mathbf{S}$ -algebra $\mathbf{A}$ validating every axiom $\mathbf{A} \models (X \vdash t = u : \star_s) \in \mathbf{Ax}$

The class of models for some presentation is called a **variety** of algebras.

Needed some technology from logic:

- ▶ **satisfaction**: $(\mathbf{A}, X \xrightarrow{e} \underline{\mathbf{A}}) \models (X \vdash t = u : \star_s)$ when: $\text{fold } \mathbf{A} \ e \ t = \text{fold } \mathbf{A} \ e \ u$.
- ▶ **validity**: $\mathbf{A} \models (X \vdash t = u : \star_s)$ when $\forall e : X \rightarrow \underline{\mathbf{A}}. (\mathbf{A}, e) \models (X \vdash t = u : \star_s)$

Example

monoids are **Monoid**-models, for the presentation $\text{Monoid} = (\text{Magma}, \mathbf{Ax}_{\text{Monoid}})$:

$$\{x, y, z\} \vdash (x \cdot y) \cdot z = x \cdot (y \cdot z) : \underline{\star} \quad \{x\} \vdash x \cdot 1 = x = 1 \cdot x : \underline{\star}$$

Recap: equational logic

The **proof theory** of algebra, the provability relation $X \vdash_{\mathcal{P}} t = u : \star_s$ is the **S**-congruence closure of the axioms:

$$\frac{}{X \vdash_{\mathcal{P}} t = t : \star_s} \text{(refl)} \qquad \frac{X \vdash_{\mathcal{P}} t = u : \star_s}{X \vdash_{\mathcal{P}} u = t : \star_s} \text{(sym)}$$

$$\frac{X \vdash_{\mathcal{P}} t = u : \star_s \quad X \vdash_{\mathcal{P}} u = v : \star_s}{X \vdash_{\mathcal{P}} t = v : \star_s} \text{(trans)}$$

$$\frac{X \vdash_{\mathcal{P}} \theta_1 = \theta_2 : Y \quad Y \vdash_{\mathcal{S}} t : \star_s}{X \vdash_{\mathcal{P}} t[\theta_1] = t[\theta_2] : \star_s} \text{(cong)} \qquad \frac{(Y \vdash t = u : \star_s) \in \mathbf{Ax}_{\mathcal{P}} \quad X \vdash_{\mathcal{S}} \theta : Y}{X \vdash_{\mathcal{P}} t[\theta] = u[\theta] : \star_s} \text{(axiom)}$$

where $X \vdash_{\mathcal{P}} \theta_1 = \theta_2 : Y$ when $X \vdash \theta_1 x = \theta_2 x : \star_r$ for all $(x : \star_r) \in Y$.

Recap: meta-theory

Theorem (soundness)

For every $\mathcal{P}$ -model $\mathbf{A}$:

$$X \vdash_{\mathcal{P}} t = u : \star_s \implies \mathbf{A} \models (X \vdash t = u : \star_s)$$

Theorem (quotiented algebra representation)

Let $\mathcal{P}$ be a presentation and X a family. We have a free $\mathcal{P}$ -model over X :

$$\mathbf{F}_{\mathcal{P}}X := \mathbf{F}_{\mathbf{S}}X / (X \vdash (=)) \quad \eta^{\mathcal{P}} : X \xrightarrow{\text{var}} \mathbf{F}_{\mathbf{S}}X \xrightarrow{[-]} (\mathbf{F}_{\mathbf{S}}X / (=)) =: \underline{\mathbf{F}_{\mathcal{P}}X}$$

I.e., for every $\mathcal{P}$ -model $\mathbf{A}$ and family map $e : X \rightarrow \underline{\mathbf{A}}$ there is a unique $\mathbf{S}$ -homomorphism extending e along $\eta^{\mathcal{P}}$.

This theorem does not help us at all to normalise expressions directly.

Satisfying $[\theta] \models (X \vdash t = u)$ in $\mathbf{F}_{\mathbf{S}}Y / (Y \vdash (=))$ amounts to proving $Y \vdash t[\theta] = u[\theta]$.

Core idea: representation and normalisation

A representation theorem for an **effective** free $\mathcal{P}$ -model over X , does help:

Theorem (extensional normalisation)

Let $(\mathbf{F}_{\mathcal{P}}X, \eta^{\mathcal{P}})$ be a free $\mathcal{P}$ -model over a family X , and $X \vdash t = u : \underline{\star}$ an equation. The following are equivalent (TFAE):

1. The unit satisfies the equation: $(\mathbf{F}_{\mathcal{P}}X, \eta^{\mathcal{P}}) \models (X \vdash t = u : \underline{\star})$
2. The given free $\mathcal{P}$ -model validates the equation: $\mathbf{F}_{\mathcal{P}}X \models (X \vdash t = u : \underline{\star})$
3. Every $\mathcal{P}$ -model $\mathbf{A}$ validates the equation: $\mathbf{A} \models (X \vdash t = u : \underline{\star})$
4. The equation is provable from $\mathcal{P}$: $X \vdash_{\mathcal{P}} t = u : \underline{\star}$

$\eta^{\text{Monoid}} := z \mapsto [z]$ makes the **effective** monoid $(\text{List}\{x, y\}, (+), [])$ **free** over $\{x, y\}$

To prove the equation $\{x, y\} \vdash_{\text{Monoid}} (x \cdot 1) \cdot y = 1 \cdot (x \cdot y) : \underline{\star}$ evaluate with η^{Monoid} :

$$([x] + []) + [y] = [x, y] = [] + ([x] + [y]) \quad \implies \quad \eta^{\text{Monoid}} \models (\{x, y\} \vdash (x \cdot 1) \cdot y = 1 \cdot (x \cdot y) : \underline{\star})$$

Summary

Core idea: algebra and normalisation

representation theorem for an **effective** algebra provides extensional **normalisation** procedure, i.e.:
decide equality

Refinement: **effective** algebra and normalisation

effective representation theorem for an **effective** algebra provides intensional **normalisation** procedure, i.e.:
decide a derivation of every equation

- ▶ Prove a constructive representation theorem for the free **setoid** $\mathcal{P}$ -model.
- ▶ Evaluate the equation in the effective free algebra, and transport it to the setoid-quotient free $\mathcal{P}$ -model.

See *Frex: dependently-typed algebraic simplification*. Guillaume Allais, Edwin Brady, Nathan Corbyn, Ohad Kammar, and Jeremy Yallop. 2025. DOI: 10.1145/3747506.

Goal

relate **normalisation** and **modern algebra**



course
URL



Zulip
URL

- ▶ Part 1: universal algebra
 - ▶ Motivation
 - ▶ Universal algebra basics: signatures & algebras
 - ▶ Modern algebra: homomorphisms & representation theorems
 - ▶ Expression normalisation: free models & equational logic
 - ▶ Modern partial evaluation: free extensions
 - ▶ Categorical algebra basics
- ▶ Part 2: second-order multi-sorted algebra
- ▶ ~~Part 3: second-order normalisation~~

Modern partial evaluation: motivation

Partially-static expressions

mixing:

- ▶ **constants**;
- ▶ algebraic operators; and
- ▶ variables

E.g., over the monoid $(\mathbb{Q}, (\cdot), 0)$:

$$-\frac{3}{4} + x + \frac{1}{4} + y + \frac{1}{2} = x + y$$

Variables can represent:

- ▶ function arguments;
- ▶ expression holes;
- ▶ delayed-evaluation place-holders; or
- ▶ sub-expressions outside the signature $\mathbf{S}$ (e.g., uninterpreted function calls)

Extension $(\mathbf{E}, s)$ of $\mathcal{P}$ -model $\mathbf{A}$

- ▶ $\mathcal{P}$ -model $\mathbf{E}$;
- ▶ $\mathcal{P}$ -homomorphism $s : \mathbf{A} \rightarrow \mathbf{E}$;

Extension homomorphism $h : (\mathbf{E}, s) \rightarrow (\mathbf{D}, r)$:

- ▶ $\mathcal{P}$ -model homomorphism $h : \mathbf{E} \rightarrow \mathbf{D}$;
- ▶ extending $r : \mathbf{A} \rightarrow \mathbf{D}$ along $s : \mathbf{A} \rightarrow \mathbf{E}$.

I.e., the **coslice category** $\mathbf{A}/\mathbf{Mod} \mathcal{P}$

Freely extending monoids

Theorem (monoid frex representation)

Let $\mathbf{A} = (\underline{\mathbf{A}}, (\cdot), 1)$ be a **monoid**, and X a set.

The **free extension (frex)** of $\mathbf{A}$ by X is given by **alternating lists**:

$$\begin{aligned}\underline{\mathbf{A}}[X] &:= \{ [a_0, x_1, a_1, \dots, x_n, a_n] \mid n \in \mathbb{N}, a_i \in \underline{\mathbf{A}}, x_i \in X \} & 0_{\mathbf{A}[X]} &:= [0_{\mathbf{A}}] \\ [a_0, x_1, a_1, \dots, x_n, a_n] \cdot_{\mathbf{A}[X]} [b_0, y_1, b_1, \dots, y_k, b_k] \\ &:= [a_0, x_1, a_1, \dots, x_n, a_n \cdot b_0, y_1, b_1, \dots, y_k, b_k] \\ s : \mathbf{A} &\rightarrow \mathbf{A}[X] & s a &:= [a] & \eta : X &\rightarrow \underline{\mathbf{A}}[X] & \eta x &:= [0_{\mathbf{A}}, x, 0_{\mathbf{A}}]\end{aligned}$$

I.e., for every extension $r : \mathbf{A} \rightarrow \mathbf{E}$ and environment map $e : X \rightarrow \underline{\mathbf{E}}$, there is a unique extension homomorphism $[r, e] : (\mathbf{A}[X], s) \rightarrow (\mathbf{E}, r)$ extending e along η , given by:

$$[r, e] [a_0, x_1, a_1, \dots, x_n, a_n] := (r a_0) \cdot_{\mathbf{E}} (e x_1) \cdot_{\mathbf{E}} (r a_1) \cdot_{\mathbf{E}} \dots \cdot_{\mathbf{E}} (e x_n) \cdot_{\mathbf{E}} (r a_n)$$

Freely extending commutative monoids

Adding commutativity $\{x, y\} \vdash x+y = y+x : \star$ simplifies the representation:

Theorem (commutative monoid frex representation)

Let $\mathbf{A} = (\underline{\mathbf{A}}, (+), 0)$ be a **commutative monoid**, and X a set.

The **free extension (frex)** of $\mathbf{A}$ by X is given by a **pairing an $\mathbf{A}$ -constant with a bag/multiset of X** :

$$\begin{aligned} \underline{\mathbf{A}[X]} &:= \underline{\mathbf{A}} \times \mathcal{M}X & 0_{\mathbf{A}[X]} &:= (0_{\mathbf{A}}, []) \\ s : \mathbf{A} &\rightarrow \underline{\mathbf{A}[X]} & s a &:= (a, \mathfrak{J}) & \eta : X &\rightarrow \underline{\mathbf{A}[X]} & \eta x &:= [0_{\mathbf{A}}, x, 0_{\mathbf{A}}] \\ (a, u) &+_{\mathbf{A}[X]} (b, v) &:= &(a+b, u \cup v) \end{aligned}$$

I.e., for every extension $r : \mathbf{A} \rightarrow \mathbf{E}$ and environment map $e : X \rightarrow \underline{\mathbf{E}}$, there is a unique extension homomorphism $[r, e] : (\underline{\mathbf{A}[X]}, s) \rightarrow (\mathbf{E}, r)$ extending e along η , given by:

$$[r, e](a, u) := (r a) +_{\mathbf{E}} \sum_{x \in u} e x$$

NB, we sum by multiplicity, e.g.: $[r, e](a, \mathfrak{J}x, x, y\mathfrak{J}) := (r a) +_{\mathbf{E}} (e x) +_{\mathbf{E}} (e x) +_{\mathbf{E}} (e y)$

Presenting extensions

Free extensions of $\mathcal{P}$ -models reduce to free models:

Evaluation presentation $\text{Eval } \mathbf{A}$ in $\mathbf{S}$ -algebra $\mathbf{A}$

- ▶ Add constants for each $\mathbf{A}$ -element: $\mathbf{S}_{\text{Eval } \mathbf{A}} := \mathbf{S} \cup \{\underline{a} : \star_s \mid a \in \mathbf{A}_s\}$
- ▶ Evaluation equation for each operator:

$$\mathbf{Ax}_{\text{Eval } \mathbf{A}} := \left\{ \begin{array}{l} \{x_1, \dots, x_n\} \vdash \\ \underline{f(\underline{a}_1, \dots, \underline{a}_n)} = \underline{f_{\mathbf{A}}(a_1, \dots, a_n)} : \star_r \mid a_i \in \mathbf{A}_{s_i} \end{array} \middle| (f : \star_{s_1} \times \dots \times \star_{s_n} \rightarrow \star_r) \in \mathbf{S}, \right\}$$

For example:

$$\left(\underline{\frac{1}{2}} + \underline{-\frac{1}{4}} \right) = \underline{\frac{1}{4}} \in \mathbf{Ax}_{\text{Eval}(\mathbb{Q}, +, 0)}$$

For $\mathcal{P}$ -model $\mathbf{A}$, the $\mathbf{A}$ -extension presentation $\mathcal{P}_{\text{Eval } \mathbf{A}}$:

$$\mathbf{S}_{\mathcal{P}_{\text{Eval } \mathbf{A}}} := \mathbf{S}_{\text{Eval } \mathbf{A}} \quad \mathbf{Ax}_{\mathcal{P}_{\text{Eval } \mathbf{A}}} := \mathbf{Ax}_{\mathcal{P}} \cup \mathbf{Ax}_{\text{Eval } \mathbf{A}}$$

Presenting extensions

There is nothing more to the theory of frex than our current development:

Theorem (extensions as models)

Let $\mathbf{A}$ be a $\mathcal{P}$ -model.

The categories of $\mathbf{A}$ -extensions and $\mathcal{P}_{\text{Eval } \mathbf{A}}$ -models are isomorphic:

$$\begin{array}{ccc} \mathbf{Ext } \mathbf{A} & \xrightarrow{\cong} & \mathbf{Mod } \mathcal{P}_{\text{Eval } \mathbf{A}} \\ & \searrow \quad \swarrow & \\ (E, s) \mapsto \underline{E} & \xrightarrow{=} & E \mapsto \underline{E} \\ & \mathbf{Fam } \mathbf{Sort} & \end{array}$$

so the frex of $\mathbf{A}$ by X is the free $\mathcal{P}_{\text{Eval } \mathbf{A}}$ -model over X .

This perspective sometimes help in constructing the frex.

Goal

relate **normalisation** and **modern algebra**



course
URL



Zulip
URL

- ▶ Part 1: universal algebra
 - ▶ Motivation
 - ▶ Universal algebra basics: signatures & algebras
 - ▶ Modern algebra: homomorphisms & representation theorems
 - ▶ Expression normalisation: free models & equational logic
 - ▶ Modern partial evaluation: free extensions
 - ▶ Categorical algebra basics
- ▶ Part 2: second-order multi-sorted algebra
- ▶ ~~Part 3: second-order normalisation~~

Categorical reformulation

In Part 2:

programs involving **object-level variables** and **substitution**

These incur 'noise'. **Abstract** from noise using **category theory**

categorical algebra dictionary

signature **S**

$\mapsto$

functor **S** : **Fam Sort** $\rightarrow$ **Fam Sort**.

algebra **A** = $((\underline{A}_s)_s, (f_{\underline{A}})_f)$ for **S**

$\mapsto$

algebra **A** = $\left(\underline{A}, \underline{S}\underline{A} \xrightarrow{(-)_{\underline{A}}} \underline{A} \right)$ for the functor **S**

variety of $\mathcal{P}$ -models

$\mapsto$

category $\mathcal{A}$

free $\mathcal{P}$ -model **F** _{$\mathcal{P}$} construction
variable embedding $\eta : X \rightarrow \underline{F_{\mathcal{P}}}$ and
homomorphic evaluation:
eval $(X \rightarrow \underline{A}) : \underline{F_{\mathcal{P}}}X \rightarrow \underline{A}$
terms and substitution

$\mapsto$

adjunction **F** $\dashv$ (**U** : $\mathcal{A} \rightarrow$ **Fam Sort**)
monad/Kleisli triple **T** = (**U** $\circ$ **F**, η , $\gg=$)

From **signatures** and algebras to **functors** and their algebras, modularly

Idea #1

Each operator with **arity** $f : \star_{s_1} \times \cdots \times \star_{s_n} \rightarrow \star_r$ yields **nodes** in the AST

- ▶ the sort of resulting **node** r
- ▶ the sort of each **sub-structure** s_i

The family $\mathbf{S}_f X$ of f -nodes with sub-structures given by $X \in \mathbf{Fam Sort}$ is:

$$(\mathbf{S}_f X)_{r'} := \begin{cases} r' = r : & X_{s_1} \times \cdots \times X_{s_n} \\ \text{otherwise} : & \emptyset \end{cases}$$

Example: **Magma** and **$\mathbb{N}$ -Quiver**

$$\underline{\mathbf{Magma}_{(\cdot)} X} := \underline{X} \times \underline{X}$$

$$(\mathbf{N}\text{-}\mathbf{Quiver}_{(\cdot)_{(m,k,n)}} X)_{m \times n} := X^{m \times k} \times X^{k \times n}$$

$$(\mathbf{N}\text{-}\mathbf{Quiver}_{(\cdot)_{(m,k,n)}} X)_{m' \times n'} := \emptyset$$

From **signatures** and algebras to **functors** and their algebras, modularly

Idea #2

An algebra $\mathbf{A}$ yields a visitor for f -nodes in the AST, folding $\mathbf{A}$ -results for substructures into results for the node:

$$f_{\mathbf{A}} : \mathbf{S}_f \underline{\mathbf{A}} \rightarrow \mathbf{A} \quad (f_{\mathbf{A}})_r(a_1, \dots, a_n) := f_{\mathbf{A}}(a_1, \dots, a_n) \quad (f_{\mathbf{A}})_{r'} \text{ is the empty function}$$

Example: magmas and matrices

$$\underline{(\cdot)}_{\mathbf{A}} : (a_1, a_2) \mapsto a_1 \cdot a_2 \quad (\cdot)_{(m,k,n), \mathbb{M}} : (M \in \mathbb{M}_{m \times k}, K \in \mathbb{M}_{k \times n}) \mapsto M \cdot K \in \mathbb{M}_{m \times n}$$

From **signatures** and algebras to **functors** and their algebras, modularly

Idea #3

The homomorphism condition:

$$h_r(f_{\mathbf{A}}(a_1, \dots, a_n)) = f_{\mathbf{B}}(h_{s_1} a_1, \dots, h_{s_n} a_n)$$

uses the **functorial action**:

$$(\mathbf{S}_f(X \xrightarrow{h} Y))_r : (\mathbf{S}_f X)_r := X_{s_1} \times \dots \times X_{s_n} \ni (x_1, \dots, x_n) \mapsto (h x_1, \dots, h x_n) \in (\mathbf{S}_f Y)_r$$

which we express as:

$$\begin{array}{ccc} (\mathbf{S}_f \underline{\mathbf{A}})_r \ni (a_1, \dots, a_n) & \xrightarrow{(\mathbf{S}_f h)_r} & (h_{s_1} a_1, \dots, h_{s_n} a_n) \in (\mathbf{S}_f \underline{\mathbf{B}})_r \\ f_{\mathbf{A},r} \downarrow & & \downarrow f_{\mathbf{B},r} \\ \underline{\mathbf{A}}_r \ni f_{\mathbf{A}}(a_1, \dots, a_n) & \xrightarrow{h_r} & h_r(f_{\mathbf{A}}(a_1, \dots, a_n)) = f_{\mathbf{B}}(h a_1, \dots, h a_n) \in \underline{\mathbf{B}}_r \end{array}$$

From **signatures** and algebras to **functors** and their algebras, modularly

Idea #3

The homomorphism condition:

$$h_r(f_{\mathbf{A}}(a_1, \dots, a_n)) = f_{\mathbf{B}}(h_{s_1} a_1, \dots, h_{s_n} a_n)$$

uses the **functorial action**:

$$(\mathbf{S}_f(X \xrightarrow{h} Y))_r : (\mathbf{S}_f X)_r := X_{s_1} \times \dots \times X_{s_n} \ni (x_1, \dots, x_n) \mapsto (h x_1, \dots, h x_n) \in (\mathbf{S}_f Y)_r$$

which we express as:

$$\begin{array}{ccc} \mathbf{S}_f \underline{\mathbf{A}} & \xrightarrow{\mathbf{S}_f h} & \mathbf{S}_f \underline{\mathbf{B}} \\ f_{\mathbf{A}} \downarrow & = & \downarrow f_{\mathbf{B}} \\ \underline{\mathbf{A}} & \xrightarrow{h} & \underline{\mathbf{B}} \end{array}$$

From **signatures** and algebras to **functors** and their algebras, modularly

Idea #4

These ideas combine using coproducts:

- ▶ Signature functor:

$$\mathbf{S}X := (\mathbf{f} : \mathbf{Op}_\mathbf{S}) \times \mathbf{S}_\mathbf{f}$$

- ▶ Algebras $\mathbf{A}$ for the signature amount to functions out of the functor at the carrier:

$$\left(\underline{\mathbf{A}}_{s_1} \times \cdots \times \underline{\mathbf{A}}_{s_n} \rightarrow \underline{\mathbf{A}}_r \right)_{(\mathbf{f} : \star_{s_1} \times \cdots \times \star_{s_n} \rightarrow \star_r) \in \mathbf{Op}} \quad \text{becomes:} \quad \mathbf{S}\underline{\mathbf{A}} \xrightarrow{(-)_\mathbf{A}} \underline{\mathbf{A}}$$

- ▶ The homomorphism condition for $h : \underline{\mathbf{A}} \rightarrow \underline{\mathbf{B}}$ amounts to the equation:

$$\begin{array}{ccc} \mathbf{S}\underline{\mathbf{A}} & \xrightarrow{\quad Sh \quad} & \mathbf{S}\underline{\mathbf{B}} \\ \downarrow \scriptstyle -_\mathbf{A} & = & \downarrow \scriptstyle -_\mathbf{B} \\ \underline{\mathbf{A}} & \xrightarrow{\quad h \quad} & \underline{\mathbf{B}} \end{array}$$

From **signatures** and algebras to **functors** and their algebras, modularly

Putting these ideas together:

Algebra $A = (\underline{A}, A_{\text{struct}})$ for functor $S : \mathcal{C} \rightarrow \mathcal{C}$

morphism $A_{\text{struct}} : S\underline{A} \rightarrow \underline{A}$.

Homomorphism of algebras $h : A \rightarrow B$ is morphism $h : \underline{A} \rightarrow \underline{B}$ satisfying:

$$\begin{array}{ccc} S\underline{A} & \xrightarrow{Sh} & S\underline{B} \\ \downarrow A_{\text{struct}} & = & \downarrow B_{\text{struct}} \\ \underline{A} & \xrightarrow{h} & \underline{B} \end{array}$$

Signature algebras are functor algebras

Theorem

Let $\mathbf{S}$ be an algebraic signature and $\bar{\mathbf{S}} : \mathbf{Fam\ Sort} \rightarrow \mathbf{Fam\ Sort}$ its signature functor. The following two categories are isomorphic:

- ▶ algebras for the signature $\mathbf{S}$ and
- ▶ algebras for the functor $\bar{\mathbf{S}}$

$$\begin{array}{ccc} \mathbf{Alg\ S} & \xrightarrow{\cong} & \mathbf{Alg\ \bar{S}} \\ & \searrow \quad \swarrow & \\ & \mathbf{Fam\ Sort} & \end{array}$$

$A \mapsto \underline{A} \quad A \mapsto \underline{A}$

Abstract syntax abstractly

Thanks to this theorem and the representation theorems for abstract syntax, we generalise:

- ▶ Algebras over a family $X \in \mathcal{C}$ amount to the **comma category**

$$X \downarrow \underline{\quad} \quad \text{objects: } (\underline{\mathbf{A}} \in \mathbf{Alg} \mathbf{S}, X \xrightarrow{d} \underline{\mathbf{A}}) \quad \begin{array}{l} \text{morphisms } (\underline{\mathbf{A}}, d) \xrightarrow{h} (\underline{\mathbf{B}}, e) : \\ \text{algebra homo. } h : \underline{\mathbf{A}} \rightarrow \underline{\mathbf{B}} \\ \text{extending } e \text{ along } d \end{array}$$

- ▶ Abstract syntax with variables from $X \in \mathcal{C}$ is the left-adjoint

$$\mathbf{F}_{\mathbf{S}} \dashv (\mathbf{Alg} \mathbf{S} \xrightarrow{\quad} \mathcal{C})$$

- ▶ The resulting monad abstracts the object of terms:

$$\begin{aligned} \mathbf{Term}_{\mathbf{S}} X &:= \underline{\mathbf{F}_{\mathbf{S}} X} & [\mathbf{var}] &:= \eta^{\mathbf{S}} : X \rightarrow \mathbf{Term}_{\mathbf{S}} X & \mathbf{eval} \mathbf{A} (X \rightarrow \underline{\mathbf{A}}) &: \mathbf{F}_{\mathbf{S}} X \rightarrow \underline{\mathbf{A}} \\ -[X \xrightarrow{\theta} \mathbf{Term}_{\mathbf{S}} Y] &:= \underline{\mathbf{eval} (\mathbf{F}_{\mathbf{S}} X) \eta^{\mathbf{S}}} : \mathbf{Term}_{\mathbf{S}} X \rightarrow \mathbf{Term}_{\mathbf{S}} Y \end{aligned}$$

Q: What role did the signature functor **S** play in the previous slide?

A: It doesn't appear there!

Varieties abstractly

- ▶ Start with any functor what-so-ever $\underline{} : \mathcal{A} \rightarrow \mathcal{C}$.
 - ▶ objects in $\mathcal{C}$ are **abstract carriers**
 - ▶ objects in $\mathcal{A}$ are **abstract models**
 - ▶ morphisms in $\mathcal{A}$ are **abstract homomorphisms**
- ▶ Algebras over a family $X \in \mathcal{C}$ amount to the comma category

$$X \downarrow \underline{}$$

- ▶ Abstract syntax with variables from $X \in \mathcal{C}$ is the left-adjoint

$$\mathbf{F} \dashv (\mathcal{A} \xrightarrow{\underline{}} \mathcal{C}) \quad \text{with} \quad (\mathbf{F}X, \eta_X) \in \mathcal{A} \text{ initial}$$

- ▶ The resulting monad abstracts the object of terms:

$$\begin{aligned} \mathbf{Term}_S X &:= \underline{\mathbf{F}_S X} & [\mathbf{var}] &:= \eta^S : X \rightarrow \mathbf{Term}_S X & \mathbf{eval} \, \mathbf{A} (X \rightarrow \underline{\mathbf{A}}) &: \mathbf{F}_S X \rightarrow \mathbf{A} \\ -[X \xrightarrow{\theta} \mathbf{Term}_S Y] &:= \underline{\mathbf{eval} (\mathbf{F}_S X) \eta^S} : \mathbf{Term}_S X \rightarrow \mathbf{Term}_S Y \end{aligned}$$

Further topics in categorical algebra

- ▶ Q: Can we **recover** all $\mathcal{P}$ -models from the free $\mathcal{P}$ -model monad $T_{\mathcal{P}} := \underline{\mathbf{F}_{\mathcal{P}}}$?

A: Always for varieties, but not for general adjunctions.

Read more about **monadic** adjunctions and **Beck's monadicity** theorem.

- ▶ Q: Which monads come from algebraic varieties?

A: **finitary** monads, as extensions of **Lawvere theories**

- ▶ Q: We worked over set-theoretic categories (**Fam Sort**).

Can we work over other categories?

A: Answers galore.

To work with **equations**, can define $\mathcal{P}$ -models in a category with **finite products**.

To **generalise** equations, many concepts bagged under **monad-theory** correspondence. Relevant buzzwords:

- ▶ Relative monads
- ▶ Monads with arities
- ▶ Enriched Lawvere theories
- ▶ Sound limit-doctrines, including parameterised algebraic theories

Goal

relate **normalisation** and **modern algebra**



course
URL



Zulip
URL

- ▶ Part 1: universal algebra
 - ▶ Motivation
 - ▶ Universal algebra basics: signatures & algebras
 - ▶ Modern algebra: homomorphisms & representation theorems
 - ▶ Expression normalisation: free models & equational logic
 - ▶ Modern partial evaluation: free extensions
 - ▶ Categorical algebra basics
- ▶ Part 2: second-order multi-sorted algebra
- ▶ ~~Part 3: second-order normalisation~~

Course structure

Goal

relate **normalisation** and **modern algebra**

- ▶ Part 1: universal algebra
- ▶ Part 2: second-order multi-sorted algebra
 - ▶ Motivation
 - ▶ Categories of contexts: finite product completion
 - ▶ Object-level and meta-level variables
 - ▶ Axiomatising object-level substitution
 - ▶ Signatures & terms
 - ▶ Equations
- ▶ ~~Part 3: second-order normalisation~~



course
URL



Zulip
URL

Motivation: from expressions to programs

Goal

treat **program** normalisation algebraically

Main distinction for us:

Object-level identifiers in terms

$$(\lambda x : \mathbb{Z}. x + 1)(y - 3) \quad \text{vs} \quad -2 + y$$

Object-level substitution in terms and equations

$$\{\mathbf{m} : (x : A \vdash \text{comp } B), \mathbf{v} : (\vdash \text{val } A)\} \vdash$$

$$(\lambda x : A. \mathbf{m}[x])(\mathbf{v}) = \mathbf{m}[x \mapsto \mathbf{v}] : \text{comp } B$$

Motivation: from expressions to programs

Goal

treat **program** normalisation algebraically

Main distinction for us:

Object-level identifiers in terms

$$(\lambda x : \mathbb{Z}. x + 1)(y - 3) \quad \text{vs} \quad -2 + y$$

Object-level substitution in terms and equations

(in e.g. Idris, we write second-order variables **m** as ?m and call them **holes**)

first-order variable
considered as a
second-order variable

$$\underbrace{\{ \mathbf{m} : (x : A \vdash \text{comp } B), \mathbf{v} : (\vdash \text{val } A) \}}_{\text{second-order variable}} \vdash \text{object-level substitution} \quad (\lambda \underbrace{x : A}_{\text{object-level identifier}}. \mathbf{m}[x])(\mathbf{v}) = \mathbf{m}[\underbrace{x \mapsto \mathbf{v}}] : \text{comp } B$$

Universe for second-order algebra

In Part 1, we organised terms with variables in X into a **family**:

$$\text{Term}_S X : \text{Sort} \rightarrow \text{Set}$$

Today, Nachi taught us that terms and their semantics are organised into **presheaves**.
Informally:

$$\text{Term}_S X : \text{Sort} \times \mathbf{Ctx}^{\text{op}} \rightarrow \text{Set} \quad \text{i.e.:} \quad \text{Term}_S X \in \mathbf{PSh}(\text{Sort} \times \mathbf{Ctx}_{\text{Sort}})$$

To make this formal, we need to define:

- ▶ category $\mathbf{Ctx}_{\text{Sort}}$
- ▶ what type of thing X is
- ▶ interface to object-level substitution
- ▶ what type of thing S is
- ▶ representation theorem for $\text{Term}_S X$

Goal

relate **normalisation** and **modern algebra**

- ▶ Part 1: universal algebra
- ▶ Part 2: second-order multi-sorted algebra
 - ▶ Motivation
 - ▶ Categories of contexts: finite product completion
 - ▶ Object-level and meta-level variables
 - ▶ Axiomatising object-level substitution
 - ▶ Signatures & terms
 - ▶ Equations
- ▶ ~~Part 3: second-order normalisation~~



course
URL



Zulip
URL

Category of contexts

Our contexts will be **structural**, i.e., support Cartesian operations:

- ▶ embedding of bindable sorts $A \mapsto x : A$;
- ▶ context **contatenation**;
- ▶ **permutation** $\Gamma, \Delta \rightarrow \Delta, \Gamma$;
- ▶ **weakening** $\Gamma, \Delta \rightarrow \Gamma$; and
- ▶ **duplication** $\Gamma \rightarrow \Gamma, \Gamma$

We use the **free finite-product completion** of the bindable sorts: $S \mapsto S_{\perp}$, it's a **category** even when S is a set. Free constructions on categories are **2-dimensional**.

Dimensions

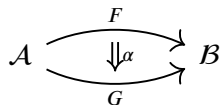
We'll need to move from 1-dimensional to 2-dimensional category theory:

- ▶ Sets are **0-dimensional**: they only have elements
- ▶ Categories are **1-dimensional**: they have objects and morphisms

To study **relationships** we typically need to move 1-dimension up.

Categories, functors, and natural transformations

Yesterday, Bob taught us the ingredients of 1-dimensional category theory:



- ▶ $\mathcal{A}$, $\mathcal{B}$ are **categories**, i.e.:
 - ▶ collections of **0-cells**: the objects $A \in \mathcal{A}$ and in $\mathcal{B}$
 - ▶ collections of **1-cells**: the morphisms $A \xrightarrow{f} B$ in $\mathcal{A}$.

with:

- ▶ identities $A \xrightarrow{\text{id}} A$
- ▶ composition $g \circ f : A \xrightarrow{f} B \xrightarrow{g} C$

such that:

every commutative diagram is a meaningful statement

Categories, functors, and natural transformations

Yesterday, Bob taught us the ingredients of 1-dimensional category theory:

$$\begin{array}{ccc} & F & \\ A & \xrightarrow{\quad} & B \\ & G & \end{array} \quad \Downarrow \alpha$$

► $F, G : \mathcal{A} \rightarrow \mathcal{B}$ are **functors**, i.e.:

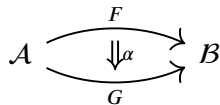
- map sending **0-cells** in $\mathcal{A}$ to **0-cells** in $\mathcal{B}$: $\mathcal{A} \ni A \mapsto FA \in \mathcal{B}$
- map sending **1-cells** in $\mathcal{A}$ to **1-cells** in $\mathcal{B}$: $(A \xrightarrow{f} B) \mapsto FA \xrightarrow{Ff} FB$
- map sending commuting squares in $\mathcal{A}$ to commuting squares in $\mathcal{B}$:

$$A \xlongequal{\quad} B \mapsto FA \xlongequal{\quad} FB$$

$$\begin{array}{ccc} & B & \\ f \nearrow & & \searrow g \\ A & \xrightarrow{\quad g \circ f \quad} & C \end{array} \quad \xlongequal{\quad} \quad \begin{array}{ccc} & FB & \\ Ff \nearrow & & \searrow Fg \\ FA & \xrightarrow{\quad F(g \circ f) \quad} & FC \end{array}$$

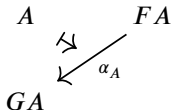
Categories, functors, and natural transformations

Yesterday, Bob taught us the ingredients of 1-dimensional category theory:

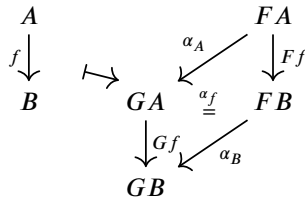


► $\alpha : F \Rightarrow G : \mathcal{A} \rightarrow \mathcal{B}$ is a **natural transformation**, i.e.:

► map sending **0-cells** in $\mathcal{A}$ to **1-cells** in $\mathcal{B}$:



► map sending **1-cells** in $\mathcal{A}$ to **2-cells** in $\mathcal{B}$:

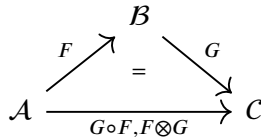


Pasting diagrams

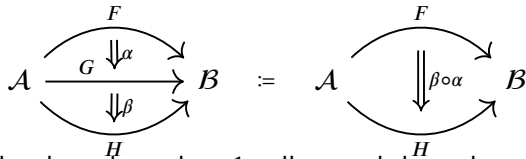
2-dimensional category theory studies 0-,1,2-cells with identities and:

- **1-cell composition:** 1-cells whose boundary 0-cells match:

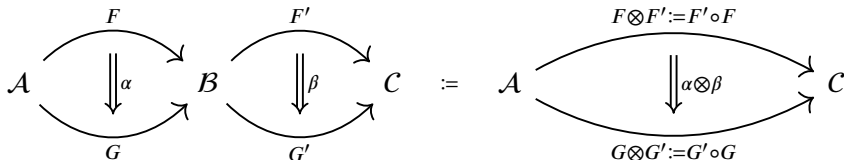
And in the next two slides we'll define:



- **2-cell vertical composition:** 2-cells whose boundary 1-cells match:



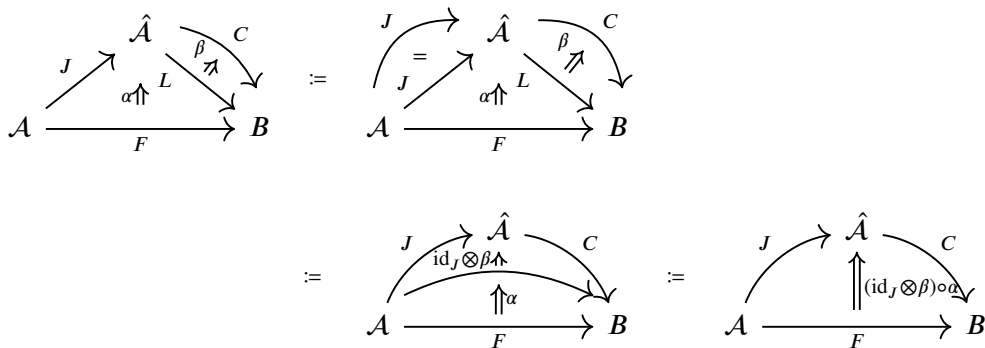
- **2-cell horizontal composition:** 2-cells whose boundary 1-cells match boundary:



such that: every pasting diagram is a meaningful 2-cell

Every pasting diagram is a meaningful 2-cell

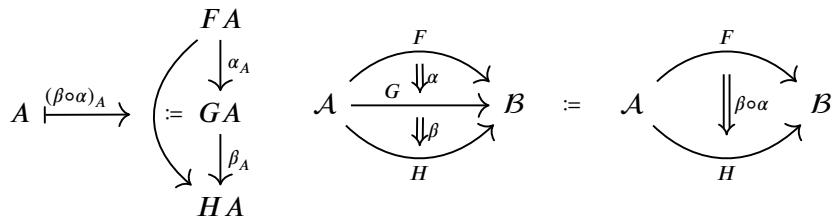
Example



The 2-category axioms (we won't cover those) ensure the resulting 2-cell does not depend on the order of 1-cell and 2-cell composition.

2-cell composition in CAT

Vertical composition



Every pasting diagram is a meaningful 2-cell

Observation: commuting diagrams

A commuting diagram equates its two sides, justifying the 'middle' 1-cell:

$$\begin{array}{ccc}
 & f & \\
 A & \xrightarrow{\quad} & B \\
 & g & \\
 & \alpha & \\
 & \text{=} & \\
 & \xrightarrow{\quad} &
 \end{array}
 \mapsto A \xrightarrow{\partial\alpha} B$$

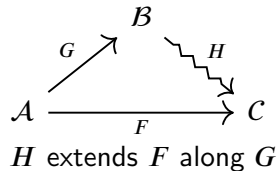
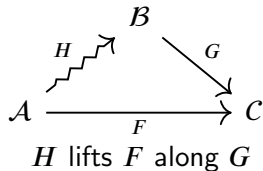
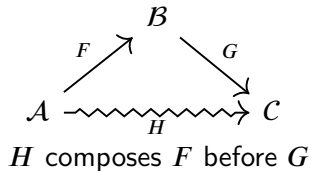
Horizontal composition

$$\begin{array}{ccc}
 \begin{array}{ccc}
 & F & \\
 A & \xrightarrow{\quad} & B \\
 & \Downarrow \alpha & \\
 & G &
 \end{array}
 &
 \begin{array}{ccc}
 & F' & \\
 B & \xrightarrow{\quad} & C \\
 & \Downarrow \beta & \\
 & G' &
 \end{array}
 &
 :=
 \begin{array}{ccc}
 & F \otimes F' := F' \circ F & \\
 A & \xrightarrow{\quad} & C \\
 & \Downarrow \alpha \otimes \beta & \\
 & G \otimes G' := G' \circ G &
 \end{array}
 \end{array}$$

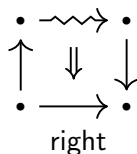
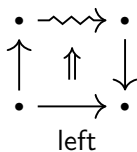
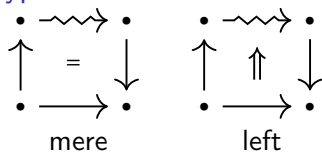
$$\begin{array}{ccc}
 A \mapsto \begin{array}{c} FA \\ \downarrow \alpha_A \\ GA \end{array} \mapsto F'GA &
 \begin{array}{ccccc}
 & F'FA & & & \\
 F'\alpha_A \swarrow & & \searrow \beta_{FA} & & \\
 & \beta_{\alpha_A} & & & \\
 & \swarrow \beta_{GA} & \nwarrow G'\alpha_A & & \\
 & G'GA & & &
 \end{array}
 &
 \begin{array}{ccc}
 F'FA = (F \otimes F')A & & \\
 \downarrow \partial\beta_{\alpha_A} & =: & \downarrow (\alpha \otimes \beta)_A \\
 G'GA = (G \otimes G')A & &
 \end{array}
 \end{array}$$

Types of 2-dimensional factorisation

Factorising 1-cells (F, G inputs; H output)



Types of factorisation



Kan

Universal

Types of 2-dimensional factorisation

Example: left Kan extension (L, ι) of F along J

a left 1-cell extension

$$\begin{array}{ccc} & \hat{A} & \\ J \nearrow & \downarrow L & \\ A & \xrightarrow{F} & B \end{array}$$

such that:

for all extensions of F along J

$$\begin{array}{ccc} & \hat{A} & \\ J \nearrow & \Downarrow \alpha & \searrow H \\ A & \xrightarrow{F} & B \end{array}$$

there exists

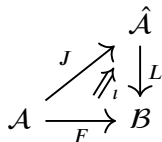
a unique 2-cell extension $\downarrow \beta \Big| \Downarrow \gamma \Big| \searrow H$ of α along ι :

$$\begin{array}{ccc} & \hat{A} & \\ J \nearrow & \Downarrow \alpha & \searrow H \\ A & \xrightarrow{F} & B \end{array} = \begin{array}{ccc} & \hat{A} & \\ J \nearrow & \Downarrow \iota & \searrow L \downarrow \beta \Big| \searrow H \\ A & \xrightarrow{F} & B \end{array}$$

Types of 2-dimensional factorisation

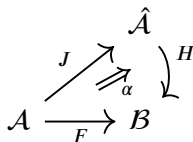
Example: left Kan extension (L, ι) of F along J

a left 1-cell extension



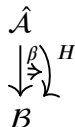
such that:

for all extensions of F along J

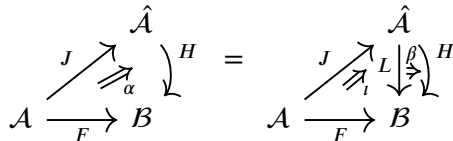


there exists

a unique 2-cell extension



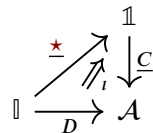
of α along ι :



Instance: colimit (C, ι) of diagram $D : \mathbb{I} \rightarrow \mathcal{A}$

left Kan extension of D along

$\star : \mathbb{I} \rightarrow \mathbb{1}$:



extensions of D
along $\star$ amount to
cocones:

an object $A \in \mathcal{A}$ and a natural
transformation $\alpha : D \rightarrow \underline{C}$.

Context category

Finite product completion $(\mathbb{A}_\perp, \mathbb{1}, (\times), J, \pi)$ of a small category $\mathbb{A}$ (informally)

- ▶ (small) category $\mathbb{A}_\perp$ which we call the **category of $\mathbb{A}$ -contexts**;
- ▶ chosen terminal object $\mathbb{1} \in \mathbb{A}_\perp$ and binary products $(\times)$;
- ▶ functor $J : \mathbb{A} \rightarrow \mathbb{A}_\perp$ which we call the **embedding** of the completion;
- ▶ for every category $\mathcal{A}$ with chosen finite products and functor $F : \mathbb{A} \rightarrow \mathcal{A}$, a choice of a right Kan extension of F along J :

$$\begin{array}{ccc} & & \mathbb{A}_\perp \\ & \nearrow J & \downarrow \pi \quad F^{\text{Env}} \\ \mathbb{A} & \xrightarrow{F} & \mathcal{A} \end{array}$$

- ▶ moreover, this right Kan extension is a **product-preserving** functor